MCSC-105 • 4 CREDITS

Discrete Mathematics and Graph Theory

Reason about finite structures, prove what must be true, and turn graph models into efficient algorithms.

STUDY BLOCKS	FORMAT	SCOPE
8	Definitions Equations Worked examples	Supplied syllabus only

How to use this handbook

For every study block: define the terms, reproduce the relation and its assumptions, solve the example without looking, and explain when the method is unsuitable.

Scope note: This handbook expands only topics named in the supplied syllabus. Confirm examination guidance, prescribed texts, and institutional policy with the responsible instructor.

Course map

1. Counting principles

Basic counting principle, Pigeonhole principle, Inclusion-exclusion principle

2. Recurrences and generating functions

Recurrence relations, Generating functions

3. Logic, sets, languages and machines

Fundamentals of logic, Set theory, Language, Finite state machines

4. Graphs, trails and cycles

Undirected and directed graphs, Modelling with graphs, Trail and cycle, Eulerian trail, Hamilton cycle

5. Connectivity and trees

Connectivity, Trees

6. Graph traversal and shortest paths

Breadth-first search (BFS), Depth-first search (DFS), Shortest path

7. Spanning trees, matching and assignment

Optimal spanning trees, Matching, Job assignment problem

8. Network flows and transportation

Optimal transportation through flows in networks

Study block 01

Counting principles

Basic counting principle	Pigeonhole principle
Inclusion-exclusion principle	

Definition and idea

The product rule multiplies the number of choices made in sequence. The pigeonhole principle guarantees repetition when more objects are placed into fewer boxes. Inclusion-exclusion corrects double counting across overlapping sets.

```
|A union B| = |A| + |B| - |A intersection B|
```

Worked example

A system generates a two-letter code followed by one digit. There are 26 x 26 x 10 = 6,760 possible codes. If 11 requests are assigned to 10 servers, at least one server receives two requests.

> **MUST UNDERSTAND**
> Can you identify whether a problem needs the sum rule, product rule, pigeonhole principle, or inclusion-exclusion?

Study block 02

Recurrences and generating functions

Recurrence relations	Generating functions

Definition and idea

A recurrence defines each term using earlier terms. A generating function encodes a sequence as coefficients of a formal power series, allowing algebraic manipulation of the whole sequence.

```
F(x) = sum(a_n x^n, n >= 0); a_n = a_(n-1) + a_(n-2)
```

Worked example

For binary strings with no consecutive 1s, let a_n be the count at length n. Splitting on the last bit gives a_n = a_(n-1) + a_(n-2), with a_1 = 2 and a_2 = 3.

> **MUST UNDERSTAND**
> Write base cases before expanding or solving a recurrence.

Study block 03

Logic, sets, languages and machines

Fundamentals of logic	Set theory
Language	Finite state machines

Definition and idea

Logic formalizes valid inference; set theory describes collections and operations. A formal language is a set of strings over an alphabet. A finite state machine accepts a string by following transitions from a start state to an accepting state.

```
p implies q is equivalent to (not p) or q
```

Worked example

A two-state machine can accept binary strings ending in 1: state q0 means the current suffix ends in 0 or is empty; q1 is accepting and means it ends in 1.

> **MUST UNDERSTAND**
> Construct a truth table and trace at least three strings through a finite state machine.

Study block 04

Graphs, trails and cycles

Undirected and directed graphs	Modelling with graphs
Trail and cycle	Eulerian trail
Hamilton cycle	

Definition and idea

A graph G=(V,E) contains vertices and edges. Directed edges have orientation. An Eulerian trail uses every edge once; a Hamilton cycle visits every vertex once and returns to its start.

```
sum(deg(v), v in V) = 2|E|
```

Worked example

A connected undirected graph has an Eulerian circuit exactly when every vertex has even degree. A delivery route that must traverse every road is an edge-focused Euler problem; visiting every city once is vertex-focused and Hamiltonian.

> **MUST UNDERSTAND**
> Do not confuse an Eulerian condition with a Hamiltonian condition.

Study block 05

Connectivity and trees

Connectivity	Trees

Definition and idea

A connected graph has a path between every pair of vertices. A tree is a connected acyclic undirected graph; with n vertices it has exactly n-1 edges and a unique simple path between each pair.

```
For a tree: |E| = |V| - 1
```

Worked example

Removing any edge from a tree disconnects it, while adding any new edge creates exactly one cycle. These dual properties are useful when proving minimal connectivity.

MUST UNDERSTAND
Prove the tree properties from connectedness and absence of cycles.

Study block 06

Graph traversal and shortest paths

Breadth-first search (BFS)	Depth-first search (DFS)
Shortest path	

Definition and idea

BFS explores by distance layers and gives shortest unweighted paths. DFS explores deeply before backtracking and supports connectivity, cycle detection, and ordering tasks. Weighted shortest paths require an algorithm whose assumptions match the edge weights.

```
BFS time = O(|V| + |E|)
```

Worked example

In an unweighted social graph, BFS from a profile discovers all profiles one connection away before those two connections away; the parent links reconstruct a shortest path.

MUST UNDERSTAND
State whether the graph is directed, weighted, and allowed to contain negative weights before choosing an algorithm.

Study block 07

Spanning trees, matching and assignment

Optimal spanning trees	Matching
Job assignment problem	

Definition and idea

A minimum spanning tree connects every vertex with minimum total edge weight. A matching contains edges with no shared endpoint. Job assignment is a weighted bipartite matching problem between workers and jobs.

```
MST has |V|-1 edges; assignment minimizes sum(c_ij x_ij)
```

Worked example

Kruskal's method repeatedly adds the lightest edge that does not create a cycle. In assignment, each worker and job must appear in exactly one selected pair.

> **MUST UNDERSTAND**
> Explain why a locally light edge is safe before applying a greedy MST step.

Study block 08

Network flows and transportation

Optimal transportation through flows in networks	

Definition and idea

A flow network is a directed graph with capacities, a source, and a sink. A feasible flow respects capacity constraints and flow conservation. Maximum flow models transportation, routing, and allocation.

```
0 <= f(u,v) <= c(u,v); inflow(v) = outflow(v)
```

Worked example

If two routes from source to sink have bottlenecks 5 and 3 and share no capacity, they can carry 8 units. An augmenting path increases flow by its smallest residual capacity.

MUST UNDERSTAND
Check capacity, conservation, and residual edges after every augmentation.

Revision checklist

Use this final pass before an assessment or interview:

- ■ I can define every listed syllabus term in plain language.
- ■ I can reproduce the central equations and state their assumptions.
- ■ I can complete each worked example and create one variation.
- ■ I can compare closely related methods and justify a choice.
- ■ I can name limitations, failure modes, and ethical or operational concerns where relevant.